函数

- C语言的构成单位是函数，一个C程序有且只有一个主函数main
- C程序永远从main开始，到main结束，与主函数main的位置无关
- 函数可以嵌套调用，但不能嵌套定义
- 主函数可以调用其他函数，其他函数之间也能相互调用，但不能调用主函数

```
void fun(){
        main();  //错误，不能调用主函数main
}
void main(){}
```

- 库函数： 必须导入相应的#include “____.h”

```
输入输出函数：  #include "stdio.h"
数学函数        #include "math.h"
字符函数        #include "ctype.h"
字符串函数      #include "string.h"
随机函数：      #include "stdlib.h" 和 #include "time.h"
```

二、自定义函数

- 有参函数定义:

  [extern] 返回数据类型 函数名(参数列表){ // 参数列表: 数据类型 参数名1, 数据类型 参数名2;
  函数体；
  }

  - extern : 函数的隐含存储类别(非常重要)
  - 每个参数数据类型和参数名必须要有
    - void fun(int a,b); X
  - 实参和形参必须一一对应(数量、类型、顺序),且数据类型必须完全一致

```
void test(int a){
     printf("a=%d",a);
}
void main()
{
   char a ='a'; //错误的赋值方法
   test(a);
}
```

- **形参数组和实参数组的长度可以不同**

```
void fun(int k[2]){
    k[1] = -100;
}

void main()
{
    int k[5] = {1,2,3,4,5};
    fun(k);  //数组参数长度可以不同
    for(int i =0;i< 5;i++){
        printf("%d  ",k[i]);  // 1  -100  3  4  5
    }
}
```

- **无参函数定义:**

  [extern] 返回数据类型 函数名(){
  函数体；
  }

- **空函数**

  void empty(){ };

三、函数的返回值

- **若函数名前是void，则函数体内一定没有return语句**

```
void main(){
    int k = fun(); //没有返回值，不能赋值
```

```
}
void fun(){};
```

- **若函数名前是int/float/double/char,则函数体内一定有return语句**
- **没有写返回值类型，将自动定义为int类型，则函数体内可能有return，也可能无返回;**
- **如果形参是基本数据类型，形参的变化不会影响实参的变化**

```
void fun(int n){
    n=n+5;
}
void main(){
    int k =30;
    fun(k);
    printf("%d",k); // k=30
}
```

- **若形参是数组或指针，若形参的值改变了，实参的值也会改变**

```
void fun(int k[]){
   for(int i=0;i<3;i++){
       k[i] =  k[i] + 1;
   }
}
void main(){
    int m[3] = {30,40,50};
    fun(m);
     for(int i=0;i<3;i++){
        printf("%d  ",m[i]);  // 31  41  51
   }
}
```

四、函数调用

- **嵌套调用**
  - **如： k=sub(subs(sub())) 或 k= sub(fun(abs()))**

- **递归调用**

三.自定义函数

3.函数的调用：

2)递归调用

如:用递归调用求某数的阶乘

```
int jc(int n)
{  if(n>=2)
     return n*jc(n-1);
   else
     return 1;
 }
main()
{ int m;
  printf("输入一个数:");
  scanf("%d",&k);
  m=jc(k);
  printf("m=%d\n",m);
 }
```

五、函数的声明

关于函数的声明：

a.若被调在前，主调在后===>不用对函数声明

```
void fun()
{ printf("***************\n"); }

void main()
{ fun(); }
```

b.若主调在前，被调在后====>必须在主调中声明

声明的方法： 把定义函数抄一遍 + ；

```
void main()
  { void fun(); //对函数的声明
    fun(); }
void fun()
   { printf("***************\n"); }
```

六、局部变量和全局变量

- 局部变量：在函数内部定义，只有当前函数范围内有效；在函数体外不能使用这些变量
    - 不同的函数体内允许可以使用同名的局部变量
- 全局变量：在函数外部定义，作用范围是从定义变量的位置开始到本文件的结束.
- 若全局变量与局部变量重名，局部变量优先全局变量(就近原则)
- 变量的完整定义形式: 存储类型 数据类型 变量名

变量的存在时间

- 静态存储 ：在变量定义时，由系统分配固定的存储空间 → 全局变量
- 动态存储: 程序运行时，由系统根据程序的需要来分配存储空间，使用完毕后立即释放空间 → 局部变量
    - 函数形参/自动变量(auto) ： 都属于局部变量，只在当前函数中有效
    - 只有定义它的函数被调用时，才会开始分配存储空间
- 变量的动态存储类型
    - auto变量： 自动变量，是一个临时存储单元，只在当前函数体内有效,存储在内存中；
        - 所有局部普通变量默认存储类型，变量的隐含存储类别是auto(非常重要)
        - 例如 int int b = 3 相当于 auto int b=3; // auto可以省略
    - register 变量： 寄存器变量，与auto变量范围相同；存放在cpu的寄存器中
        - 对register变量不能进行取地址的运算

```
register int k2;
scanf("%d",&k2);  //错误
```

        - 只有局部自动变量和形式参数才可以定义为寄存器变量
        - 属于动态存储方式
        - (非常重要)只有在使用时才为该类型变量分配内存的存储类说明是： auto 和 register
- 变量的静态存储类型 （高频考点）
    - static 静态局部变量(在函数体定义）
        - 可以不必赋初始值，系统自动赋值0

- **作用域和自动变量相同**
- **永久性存储(生命周期为整个程序，退出函数体也不会消失)**
- **只在第一次调用时初始化**
- **值可以延伸(重要)**

```
//不加static，三个fun()都输出m=52
// 加static ，分别输出： m=52 ,m=54 , m=56
void fun(){
    static int m =50;    // 加static后，每次都保留最新的m值
    m+=2;
    printf("m=%d\n",m);
}
void main()
{
   fun();
   fun();
   fun();
}
```

错题：

1. **普通局部变量必须赋值，否则输出报错**

```
void main()
{
   int k;
   printf("k=%d\n",k);  // 程序报错
}

------------------------------
void main()
{
   static int b;   // 用static定义的变量,没有赋值情况下，系统默认是0
   printf("b=%d\n",b);  //输出0
}
```